

Documentation

API Endpoints

Authentication

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Register a new company account. Returns company data and a JWT valid for 7d
POST	/api/auth/login	No	Login with email and password. Returns a JWT on success.

Jobs

Method	Endpoint	Description
POST	/api/jobs	Create a new job posting. Costs 1 credit per job.
PATCH	/api/jobs/:id	Update a job. Only the owner company can update.
GET	/api/jobs	Search and filter job listings. Supports text search, filtering, sorting, and pagination.

Search Query Parameters

Parameter	Type	Default	Description
search	string	–	Full-text search on title, description, skills
location	string	–	Filter by job location
industry	string	–	Filter by industry
jobType	string	–	internship, fullTime, partTime
workplaceType	string	–	onsite, remote, hybrid
status	string	available	available or completed
sortBy	string	newest	newest, deadline, views
page	number	1	Page number
limit	number	10	Results per page (max 50)

Error Responses

All errors follow a consistent format: `{ "success": false, "msg": "..."}`

Status	Code	When
400	Bad Request	Joi validation failure – missing required fields, invalid data types, or enum mismatch.
401	Unauthorized	Missing or invalid JWT token. Also returned for wrong email/password on login.
403	Forbidden	No remaining job credits, or attempting to update a job owned by another company.
404	Not Found	Job ID does not exist in the database.
409	Conflict	Duplicate email on registration.
500	Server Error	Unexpected internal error. The actual error message is hidden from the client.

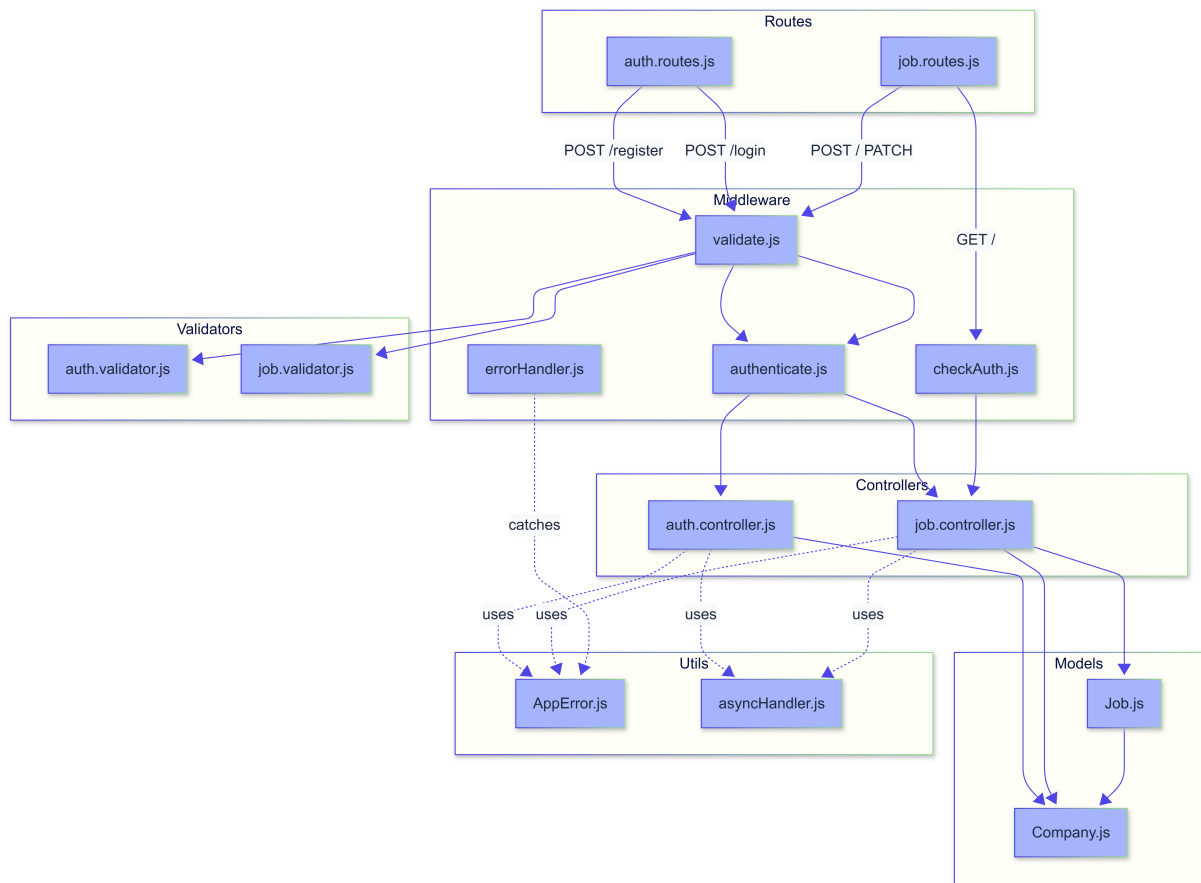
Project Structure

```

starter/
  config/           Database connection setup
  controllers/      Request handlers (auth, jobs)
  middleware/       authenticate, checkAuth, validate, errorHandler
  models/           Mongoose schemas (Company, Job)
  routes/           Express route definitions
  utils/            AppError class, asyncHandler wrapper
  validators/       Joi validation schemas
  requests.rest     Pre-built API test requests
  app.js            Application entry point

```

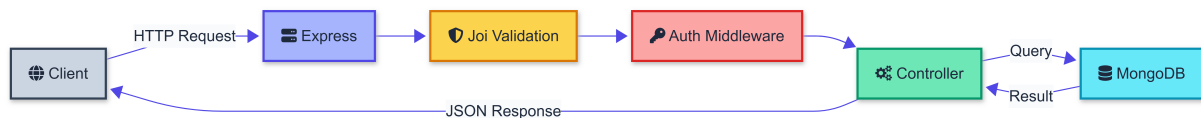
- **utils/AppError.js** – Custom error class that carries an HTTP status code, allowing the error handler to return meaningful responses.
- **utils/asyncHandler.js** – Wraps async route handlers to automatically catch errors and pass them to `next()`, eliminating repetitive try/catch blocks.
- **validators/** – Joi schemas for input validation. The `updateJobSchema` is built from `createJobSchema` using `.fork()` to avoid code duplication (DRY). Dates are validated with `.iso()`, `.greater('now')`, and `.min(Ref)`.
- **middleware/authenticate.js & checkAuth.js** – `authenticate` enforces auth and throws 401 on failure. `checkAuth` silently attaches user data if a valid token is present, without blocking unauthenticated access.



Architecture & Design Decisions

Request Flow

Every incoming request passes through the following pipeline:



1. **Joi Validation** – checks data types, required fields, enum values. Rejects invalid input with a 400 response.
2. **Auth Middleware** – verifies the JWT and confirms the company still exists in the database.
3. **Controller** – executes the business logic and responds.

Database Schema

COMPANY			
String	email	UK	unique lowercase
String	password		hashed - select false
String	role		default: company
String	companyName		required
String	companyIndustry		optional
String	companyLocation		optional
Number	companyJobCredits		default: 5

posts (1:N)

JOB			
ObjectId	company	FK	ref Company
String	jobTitle		required - text indexed
String	jobType		enum: internship fullTime partTime
String	jobWorkPlaceType		enum: onsite remote hybrid
String	jobStatus		enum: available completed
String	jobDescription		text indexed
String	jobLocation		compound indexed
String	jobIndustry		compound indexed
Date	jobApplicationDeadline		required
Array	jobSkills		text indexed
Number	jobViewCount		default: 0

Testing

The project includes a `requests.rest` file with pre-built API requests covering both success and failure scenarios. Open it in VS Code with the **REST Client** extension and click “Send Request” above each block.

Covered cases:

- Registration with valid data, empty body, and duplicate email detection (409)
- Login with correct and incorrect credentials (401)
- Job creation with and without authentication (401)
- Job creation with invalid enum values (400)
- Job creation with deadline in the past (400 – date validation)
- Job update with and without token (401)
- Job update with ownership verification – cross-company rejection (403)
- Search with filters, text search, and pagination edge cases